

Report Esercizio Python – Calcolo Perimetro e Area

Esercizio Base

In questo esercizio ho scritto un programma in Python che permette di calcolare il perimetro e l'area di una figura geometrica tra quadrato, rettangolo e cerchio.

All'avvio, il programma mostra un piccolo menu con la lista delle figure e chiede all'utente di selezionarne una inserendo un numero (1, 2 o 3). Dopo la scelta, il programma chiede i dati necessari in base alla figura scelta (es. lato, base, altezza o raggio), poi fa i calcoli e mostra a schermo i risultati.

Tutti i calcoli e i risultati sono mostrati in modo chiaro nel terminale. Il codice è semplice, utilizza solo `input()`, `float()`, `print()` e strutture `if`, come spiegato nelle slide viste in classe.

```
GNU nano 8.2                                eserciziow6d5.py
^wd65
print("=== Calcolo perimetro e area ===")
print("1. Quadrato")
print("2. Rettangolo")
print("3. Cerchio")
scelta = input("Scegli una figura (1/2/3): ")

# Quadrato
if scelta == "1":
    lato = float(input("Inserisci il lato del quadrato: "))
    perimetro = 4 * lato
    area = lato * lato
    print("Perimetro:", perimetro)
    print("Area:", area)

# Rettangolo
elif scelta == "2":
    base = float(input("Inserisci la base del rettangolo: "))
    altezza = float(input("Inserisci l'altezza del rettangolo: "))
    perimetro = 2 * (base + altezza)
    area = base * altezza
    print("Perimetro:", perimetro)
    print("Area:", area)

# Cerchio
```

```
# Cerchio
elif scelta == "3":
    raggio = float(input("Inserisci il raggio del cerchio: "))
    pi = 3.14
    perimetro = 2 * pi * raggio
    area = pi * raggio * raggio
    print("Perimetro:", perimetro)
    print("Area:", area)
```

```
# Errore
else:
    print("Scelta non valida")
```

```
—(kali@kali)-[~]
—$ python3 eserciziow6d5.py
=== Calcolo perimetro e area ===
1. Quadrato
2. Rettangolo
3. Cerchio
Scegli una figura (1/2/3): 3
Inserisci il raggio del cerchio: 5
Perimetro: 31.400000000000002
Area: 78.5
```

```
—(kali@kali)-[~]
—$ python3 eserciziow6d5.py
=== Calcolo perimetro e area ===
1. Quadrato
2. Rettangolo
3. Cerchio
Scegli una figura (1/2/3): 1
Inserisci il lato del quadrato: 2
Perimetro: 8.0
Area: 4.0
```

```
—(kali@kali)-[~]
—$ python3 eserciziow6d5.py
=== Calcolo perimetro e area ===
1. Quadrato
2. Rettangolo
3. Cerchio
Scegli una figura (1/2/3): 2
Inserisci la base del rettangolo: 2
Inserisci l'altezza del rettangolo: 4
Perimetro: 12.0
Area: 8.0
```

Conclusione –Esercizio Prima Parte

Questo esercizio mi ha aiutato a capire meglio come si scrive un programma in Python. Ho usato `input()` per far inserire i dati all'utente e `if, elif` per fare le scelte. Con `print()` ho mostrato i risultati. Ho calcolato perimetro e area in modo semplice. È un po' come in C++, ma Python è più facile da scrivere e da leggere.

Parte opzionale dell'esercizio

In questo esercizio ci hanno dato una traccia che all'inizio sembrava solo una versione più lunga e complicata di quella base, ma poi si è rivelata una specie di tranello tecnico. Il prof, senza dirci troppo, ci ha fatto lavorare su un codice volutamente pieno di piccoli errori, alcuni più evidenti, altri più nascosti. Il bello (e brutto) è che all'inizio sembrava tutto giusto, il codice funzionava, dava risultati, e non uscivano nemmeno errori nel terminale. Però andando avanti mi sono accorto che le cose non quadravano, soprattutto confrontando i risultati e ragionando meglio sulle formule usate. Tutto questo mi ha fatto capire che non basta scrivere codice che funziona: bisogna davvero capire cosa si sta facendo riga per riga. E infatti appena abbiamo analizzato il codice con più attenzione, sono venuti fuori un sacco di problemi di logica, variabili usate male, valori messi a caso e anche errori nei nomi delle cose. Questo esercizio mi ha fatto capire davvero l'importanza di leggere bene la traccia, controllare la logica e scrivere in modo più ordinato.

All'inizio sembrava un'estensione normale dell'esercizio base, ma in realtà la parte opzionale era scritta male apposta. Il prof voleva che ci accorgessimo degli errori nascosti. E io, riga per riga, ho iniziato a notare tutto quello che non funzionava.

Tabella errori trovati

Riga/e	Errore	Spiegazione a modo mio
52, 54, 56	Valori salvati nella stessa variabile	Il prof ci ha fregati con sta cosa: perimetro e area vengono messi entrambi in <code>valore_iniziale</code> , ma così il perimetro si perde subito. Andavano salvati in due variabili diverse.
54	Valori messi a caso tipo <code>lato2 = valore_iniziale / 2</code>	Questa cosa non ha una logica, viene fuori solo per far girare il programma. Se il valore iniziale è strano, i risultati non hanno senso. Andava chiesto all'utente o fatto meglio.
6 (e altre)	Uso confuso tra <code>3.14</code> e <code>math.pi</code>	In certi punti uso <code>3.14</code> , in altri <code>math.pi</code> . Sta cosa è sbagliata, o si usa uno o si usa l'altro. Meglio <code>math.pi</code> che è più preciso. Basta scrivere <code>import math</code> e via.
27	Variabili con nomi sbagliati o poco chiari	Uso <code>lato1</code> per indicare il raggio nel cerchio, ma non si capisce niente così. Serve usare i nomi giusti tipo <code>raggio</code> , sennò si fa solo confusione leggendo il codice.
52 e seguenti	Uso dell'area come nuovo valore iniziale	L'idea di usare l'area come valore per la figura dopo può sembrare figa, ma non ha senso. L'area non rappresenta né un lato né un raggio, quindi i conti diventano strani.

Descrizione:

Una delle cose che mi ha fatto capire che il codice aveva problemi è stato quando ho visto che i valori restituiti dalla funzione venivano salvati male (riga 52, 54, 56). In pratica, invece di salvare perimetro e area in due variabili diverse, venivano messi entrambi in una sola, sempre `valore_iniziale`. Così il perimetro veniva praticamente buttato via, perché appena salvato veniva sovrascritto dall'area. Il programma andava avanti lo stesso, ma alla fine stavo usando solo l'area per continuare i calcoli. E questa cosa è sbagliata, soprattutto perché la traccia diceva chiaramente che dovevo usare entrambi i valori. Bastava scrivere per bene due variabili diverse e tutto si sarebbe risolto.

Poi ho notato che alcuni valori venivano scelti un po' a caso (es. riga 54 per il rettangolo: `lato2 = valore_iniziale / 2`). Per esempio, per figure come il rettangolo, si faceva una divisione per due per ottenere un'altezza, ma non c'è nessuna regola geometrica che lo giustifica. Se il valore iniziale è 20, allora magari i conti escono "bene", ma se è 7 o 13, i risultati non hanno proprio senso. È come dire: "boh, dividiamolo a metà e speriamo che vada bene".

Un errore che all'inizio non avevo nemmeno notato è quello **del pi greco usato in due modi diversi** (`math.pi` in riga 6, mentre 3.14 viene messo a mano altrove). Alcune volte si scrive proprio `pi = 3.14`, in altre invece si usa `math.pi`. Questa cosa può portare a risultati diversi anche se il valore iniziale è lo stesso, e fa perdere precisione. Oltre a essere confusionaria. La cosa più corretta sarebbe usare `math.pi` ovunque, ma solo se ho fatto `import math` all'inizio (che in alcuni casi mancava pure). Quindi questo è un errore anche di incoerenza e precisione.

Poi c'è una cosa che mi ha fatto confondere, cioè **i nomi delle variabili poco chiari** (come `lato1` usato al posto di `raggio`, tipo nella funzione `cerchio` in riga 27). Se sto lavorando su un cerchio, la variabile dovrebbe chiamarsi `raggio`, non `lato1`. Perché `lato1` va bene per un quadrato o un rettangolo, ma nel cerchio non ha senso. Quando leggi il codice è più difficile capire cosa sta succedendo se i nomi non sono chiari. Magari funziona tutto, ma non si capisce niente. E se un altro lo legge, si perde.

Infine c'è il fatto che **l'area viene usata come nuovo valore iniziale** per la figura successiva (succede dopo ogni calcolo dentro il ciclo, quindi riga 52 e seguenti). Questa cosa a volte funziona per puro caso, ma non ha una vera logica dietro. L'area non può essere sempre riutilizzata come se fosse una lunghezza o un lato per un'altra figura. Cioè: se calcolo l'area di un quadrato e poi la uso come raggio di un cerchio, i risultati non hanno più senso. È una scelta fatta solo per far andare avanti il programma, ma non è corretta dal punto di vista logico.

Correzione del programma

La prima cosa che ho deciso di sistemare è stata quella parte dove si salvavano i risultati della funzione. Prima venivano messi tutti dentro `valore_iniziale`, sia il perimetro che l'area, ma così facendo il primo valore andava perso subito. Allora ho fatto in modo che si salvassero in due variabili diverse, così non si butta via niente e si possono usare separatamente, come si deve.

Poi ho sistemato anche il discorso del pi greco. In alcune parti del programma c'era scritto 3.14, in altre `math.pi`. Sta cosa non va bene, perché o si usa uno o l'altro, sennò i risultati escono diversi. Alla fine ho deciso di usare solo `math.pi`, che è più preciso, e ovviamente ho aggiunto all'inizio `import math` per poterlo usare.

Un'altra cosa che mi dava fastidio erano i nomi delle variabili, che non erano chiari. Tipo per il cerchio c'era scritto `lato1`, ma non ha senso, il cerchio non ha lati. Così l'ho chiamato `raggio`, così si capisce al volo. Per il quadrato invece ho lasciato `lato`, che è giusto e non crea confusione.

Poi c'era il problema dei valori inventati. In certi punti il programma prendeva il valore iniziale e lo divideva o lo modificava per tirare fuori altri dati. Però senza logica, tipo "dividiamo a metà e vediamo che succede". Sta cosa l'ho tolta, e adesso ogni figura chiede direttamente all'utente i dati che servono. Almeno così i numeri hanno senso.

Infine ho tolto anche l'idea di usare l'area come valore per la figura successiva. Non ha senso passarsi l'area come se fosse un lato o un raggio, sennò i conti si sballano. Adesso ogni figura si calcola da sola, con i suoi dati, e non dipende più da quello che è successo prima.

Ora questo è il codice che ho usato per il nuovo programma

```
GNU nano 8.2                                eserciziow6d5opzionale.py
import math

def calcola_perimetro_area(figura):
    if figura == "Quadrato":
        lato = float(input("Inserisci il lato del quadrato: "))
        if lato <= 0:
            print("Valore non valido.")
            return 0, 0
        perimetro = lato * 4
        area = lato * lato

    elif figura == "Rettangolo":
        base = float(input("Inserisci la base del rettangolo: "))
        altezza = float(input("Inserisci l'altezza del rettangolo: "))
        if base <= 0 or altezza <= 0:
            print("Valori non validi.")
            return 0, 0
        perimetro = (base + altezza) * 2
        area = base * altezza

    elif figura == "Cerchio":
        raggio = float(input("Inserisci il raggio del cerchio: "))
        if raggio <= 0:
            print("Valore non valido.")
            return 0, 0

    else:
        print("Figura non trovata.")
        return 0, 0

    print("Perimetro di", figura, ":", perimetro)
    print("Area di", figura, ":", area)
    return perimetro, area

figure_disponibili = ["Quadrato", "Rettangolo", "Cerchio"]
fine_programma = False

while fine_programma == False and len(figure_disponibili) > 0:
    print("\nFigure disponibili:")
    for i in range(len(figure_disponibili)):
        print(i + 1, "-", figure_disponibili[i])

    scelta = input("Scegli il numero della figura oppure scrivi 'fine' per uscire: ")

    if scelta == "fine":
        print("Hai scelto di uscire dal programma.")
        fine_programma = True
    else:
```



```

# Tenta la conversione in numero
try:
    numero = int(scelta)
    if numero >= 1 and numero <= len(figure_disponibili):
        figura_scelta = figure_disponibili[numero - 1]
        perimetro, area = calcola_perimetro_area(figura_scelta)
        figure_disponibili[numero - 1] = "Già usata"
    else:
        print("Numero non valido.")
except:
    print("Inserimento non valido. Scrivi un numero valido oppure 'fine'.")

# Controlla se tutte le figure sono state usate
tutte_usate = True
for f in figure_disponibili:
    if f != "Già usata":
        tutte_usate = False

if tutte_usate:
    print("\nHai usato tutte le figure. Programma completato.")
    fine_programma = True

```

*Stavo cercando un modo per controllare se l'utente scriveva un numero o una parola, tipo quando deve scegliere il numero della figura. All'inizio avevo pensato di usare `isdigit()`, ma poi mi sono ricordato che nelle slide questo comando non lo abbiamo ancora fatto.

Allora sono andato a cercare un attimo su internet e ho trovato questa cosa chiamata `try` e `except`.

L'ho capita subito perché è molto semplice: praticamente dice al programma "prova a trasformare quello che ha scritto l'utente in un numero, e se non ci riesci non dare errore, ma stampa solo un messaggio tipo che non è valido".

Questa cosa mi è sembrata utile perché se uno scrive per sbaglio "ciao" o qualcosa che non è un numero, il programma non si blocca e continua tranquillamente. Quindi ho deciso di usarla nel mio codice anche se non l'abbiamo ancora vista a lezione, perché era la cosa più semplice per risolvere il problema in quel momento.

Prova Del Programma

```

(kati@kati)-[~]
$ python3 eserciziow6d5opzionale.py
Figure disponibili:
1 - Quadrato
2 - Rettangolo
3 - Cerchio
Scegli il numero della figura oppure scrivi 'fine' per uscire: 1
Inserisci il lato del quadrato: 2
Perimetro di Quadrato : 8.0
Area di Quadrato : 4.0

Figure disponibili:
1 - Già usata
2 - Rettangolo
3 - Cerchio
Scegli il numero della figura oppure scrivi 'fine' per uscire: 2
Inserisci la base del rettangolo: 2
Inserisci l'altezza del rettangolo: 4
Perimetro di Rettangolo : 12.0
Area di Rettangolo : 8.0

Figure disponibili:
1 - Già usata
2 - Già usata
3 - Cerchio
Scegli il numero della figura oppure scrivi 'fine' per uscire: 3
Inserisci il raggio del cerchio: 2
Perimetro di Cerchio : 12.566370614359172

```

Conclusione

Alla fine posso dire che questo esercizio facoltativo mi ha fatto ragionare molto più del previsto. Quando ho iniziato pensavo che il programma dato nelle slide fosse giusto, invece mentre lo analizzavo ho iniziato a notare che c'erano delle cose che non tornavano. Piano piano ho trovato diversi errori, tipo che i valori venivano salvati nella stessa variabile, che si usava sia `3.14` che `math.pi`, e che alcune variabili avevano nomi poco chiari, tipo chiamare raggio "lato1".

Ho capito che anche se un programma non dà errori, non vuol dire che è scritto bene. Per questo ho deciso di riscriverlo da capo con i comandi che conosco, usando solo quelli delle slide, e cercando di fare tutto in modo più chiaro e semplice.

Mi sono messo lì con calma, ho pensato bene a cosa cambiare e a come farlo capire anche a chi legge il codice.

Facendo questo esercizio è che prima di iniziare a scrivere un programma è fondamentale leggerselo tutto con calma. Se uno parte subito a scrivere senza capire bene cosa succede riga per riga, poi rischia di fare errori che non si vedono subito ma che rovinano tutto il resto. Infatti io all'inizio pensavo che il programma dato fosse giusto, poi solo guardandolo con attenzione ho visto che c'erano diversi problemi, anche piccoli, ma che alla fine incasinavano i conti.

Un'altra cosa importante è che quando non capivo qualcosa o mi serviva una soluzione, ho imparato che cercare online aiuta davvero. Ad esempio il blocco `try-except` non lo avevamo ancora visto nelle slide, ma l'ho scoperto cercando e mi ha risolto un problema nel modo più semplice. In programmazione secondo me non si può sapere tutto a memoria, ma l'importante è imparare a ragionare, capire dove sta il problema e trovare un modo per risolverlo.

Quindi questo esercizio mi ha fatto capire che per scrivere bene un programma non basta scrivere codice, ma bisogna capirlo, controllarlo e saper cercare quando serve. Questo è il modo giusto per migliorare e non fare errori banali.

Andrea Di Rocco